01

まずは「処理の流れ」を読む

情報処理安全確保支援士 科目B（旧・午後）｜JavaScriptコード読解 01

試験での目標：コードを書き上げることではなく、**入力がどこから来て、どの処理を通り、どこへ出力されるか**を追えること。細かな文法暗記より、データフローと信頼境界を優先する。

1. 最低限の文法

変数・値・比較

- `const`：再代入しない変数。オブジェクト内部の変更までは禁止しない。
- `let`：再代入する変数。`var`は古いコードで登場する。
- `===`／`!==`：型を変換せず厳密に比較。まずこちらで読む。
- `&&`、`||`、`!`：AND、OR、NOT。
- `?.`：途中が`null`等なら停止。`??`：`null`等のとき既定値。

配列・オブジェクト

- `a[0]`：配列の先頭。`user.name`：プロパティ参照。
- `map`：変換、`filter`：絞込み、`find`：最初の一致。
- `{ id, role }`：同名プロパティの取り出し。
- `JSON.parse(s)`：文字列→値。`JSON.stringify(v)`：値→文字列。
- APIから得たJSONも、内容を信用してよいとは限らない。

```
const users = response.items;                // 配列
const active = users.filter(u => u.enabled);  // enabled が真の要素
const admin = active.find(u => u.role === "admin");
if (admin?.name) {
  nameArea.textContent = admin.name;          // 文字として表示
}
```

読み方　①代入先に丸を付ける → ②条件式の真偽を確認 → ③値が変化する箇所だけ追う。配列の全要素を頭の中で展開しない。

2. 関数・イベント・非同期処理

関数

`function f(x){...}`または`x => ...`。引数が入力、`return`が出力。関数名より「何を受けて何を返すか」を見る。

イベント

`addEventListener`は、クリックやメッセージ受信を起点に処理する。`event.target`や`event.data`は入力源になり得る。

非同期

`await`は結果を待つ。`Promise.then()`も同系統。通信結果を受け取った後の処理を追う。

```
async function loadProfile(id) {
  const res = await fetch(`/api/users/${encodeURIComponent(id)}`);
  if (!res.ok) throw new Error("request failed");
  const data = await res.json();       // 外部由来の値
  profile.textContent = data.displayName;       // HTMLとして解釈しない
}
```

後回しでよいもの：高度なクラス設計、ビルドツール、フレームワーク固有の状態管理、複雑な型システム。試験対策では、短いコード片の入力・条件分岐・通信・出力を読めればよい。

SOURCE → TRANSFORM → SINK

02

ブラウザ内の「入口」と「出口」

脆弱性は、信用できないデータが危険な出口へ届くときに生まれる

SOURCE / 入力源
URL、フォーム、Cookie、Storage、API応答、postMessage

→

TRANSFORM / 処理
検証、変換、連結、デコード、権限判定

→

SINK / 出力先
DOM、URL、SQL、OSコマンド、ログ、通信

3. DOM操作：XSSを見抜く

コード／API	意味	試験での着眼点
`textContent` `createTextNode`	文字列として表示	基本は安全側 HTMLとして解釈させない。

`innerHTML`
`outerHTML`
`insertAdjacentHTML`
	HTMLとして挿入	危険なsink 外部入力が届けばDOM-based XSSを疑う。

`eval`
`Function`
`setTimeout("...")`
	文字列をコードとして実行	外部入力を渡さない。原則として利用を避ける。

`setAttribute`	属性を設定	属性次第。`href`／`src`はURLスキーム、`on*`はイベント実行に注意。
`location.href` `window.open`	別URLへ移動	許可先を検証しないとオープンリダイレクト。`javascript:`等にも注意。

```
// 脆弱：q がHTMLとして解釈される
const q = new URLSearchParams(
  location.search
).get("q");
result.innerHTML = `検索語: ${q}`; // NG
```

```
// 対策：文字として挿入する
const q = new URLSearchParams(
  location.search
).get("q");
result.textContent = `検索語: ${q}`; // OK
```

「エスケープする」だけでは不十分。 HTML本文、属性、URL、JavaScript文字列では必要な処理が異なる。まず不要なHTML解釈をやめ、出力先に合ったAPIを使う。URLのクエリ値には`encodeURIComponent`を使うが、これはHTML出力のXSS対策とは別物。

4. URL・Cookie・通信

オリジンとCORS

- オリジン＝**スキーム＋ホスト＋ポート**。
- 同一オリジンポリシーは、主に他オリジンの応答を読むことを制限する。
- CORSはサーバが読取りを許可する仕組み。`Access-Control-Allow-Origin`を入力値のまま返す実装は要注意。
- CORSは認証・認可の代わりではない。

Cookieと保存領域

- `HttpOnly`：JavaScriptからCookieを読ませない。
- `Secure`：HTTPS通信時だけ送信。
- `SameSite`：クロスサイト送信を制御し、CSRFを軽減。
- `localStorage`はJavaScriptから読める。認証秘密の保存先として慎重に扱う。

```
window.addEventListener("message", event => {
  if (event.origin !== "https://trusted.example") return; // 送信元を確認
  status.textContent = event.data.message;                // 文字として表示
});
```

混同注意　**XSS**はスクリプト実行、**CSRF**は利用者の認証状態を悪用した意図しない操作、**CORS**は異なるオリジンからの応答読取り制御。対策を取り違えない。

脆弱コードを診断する手順

設問では「原因」「攻撃成立条件」「影響」「対策」を分けて答える

5. 頻出パターン早見表

論点	コードから探すもの	対策の方向
XSS	URL・API値 → `innerHTML`、文字列からのコード実行	`textContent`、文脈別エンコード、必要時のHTMLサニタイズ、CSPによる多層防御
CSRF	Cookie認証＋状態変更API、トークンや送信元確認なし	CSRFトークン、`SameSite`、`Origin`確認。GETで状態変更しない
認可不備	画面でボタンを隠すだけ、要求されたIDをそのまま検索	サーバ側で「その利用者が対象データを操作可能か」を毎回確認
CORS不備	要求の`Origin`を無条件反映、資格情報付きで広く許可	許可オリジンの固定リスト、必要最小限のメソッド・ヘッダー
リダイレクト	`location`や`next`に外部入力を使用	相対パス限定、許可ホストの照合、危険なURLスキーム拒否
postMessage	`event.data`を無条件利用、送信時の`"*"`	受信時に`event.origin`・データ形式を検証、送信先を限定
秘密情報	トークンをURL・DOM・Storage・ログへ出力	露出範囲を最小化。Cookieなら`HttpOnly`等を適切に設定

6. 8ステップ読解法

1. **主体を確認：**利用者、攻撃者、ブラウザ、各サーバを区別する。
2. **信頼境界を引く：**外部入力、別オリジン、別権限をまたぐ箇所。
3. **入力源を囲む：**URL、フォーム、Cookie、API応答、メッセージ。
4. **値を追う：**代入、連結、デコード、条件分岐、関数の引数。
5. **危険な出口を探す：**DOM、URL、SQL、コマンド、ログ、外部送信。
6. **成立条件を言語化：**誰が何を制御できると攻撃が成立するか。
7. **影響を特定：**閲覧、改ざん、なりすまし、情報窃取など。
8. **原因に効く対策を書く：**抽象語でなく、検証場所・API・設定まで示す。

7. ミニ演習：何が安全で、何が未解決か

```
const id = new URLSearchParams(location.search).get("id");
const res = await fetch(`/api/document?id=${encodeURIComponent(id)}`);
const doc = await res.json();
title.textContent = doc.title;
if (doc.canDelete) deleteButton.hidden = false;
```

コードから言える安全側の点

- IDはURL部品としてエンコードされている。
- タイトルは`textContent`で表示され、HTMLとして解釈されない。

このコードだけでは保証できない点

- サーバがIDの形式を検証しているか。
- 閲覧・削除権限をサーバ側でも確認しているか。
- ボタンを隠すだけでは認可対策にならない。

答案の型　「外部から指定された**［入力］**を検証せず**［危険な処理］**に使用しているため、攻撃者が**［成立条件］**を満たすと**［影響］**が生じる。サーバ側で**［具体的対策］**を行う。」

直前チェック

読めるか

`const/let`、配列・オブジェクト、関数、イベント、`fetch/await`。

区別できるか

XSS／CSRF／CORS、認証／認可、URLエンコード／HTMLエンコード。

答えられるか

入力源、危険な出口、攻撃成立条件、影響、原因に対応した具体策。

試験制度メモ：2026年度からCBT方式となり、従来の「午後試験」は「科目B試験」に名称変更。IPAは、問う知識・技能の範囲、記述式という出題形式、出題数・解答数等は変更しないと案内している。参考：IPA「情報処理安全確保支援士試験」関連資料（2026年9月確認）。